■ SUDARSHAN STUDIO (■■■■■■■ ■■■■■■■■)

System Architecture, Tech Stack, ML Foundations & Vibe Coding Guide

Executive Summary: Sudarshan Studio is an intelligent AI-powered visual system whose *behaviour is the message*. Unlike generic image generators that produce disconnected outputs, Sudarshan Studio is an integrated multimodal platform where human emotion, visual context, deep salient object matting, prompt-driven optical enhancement, and acoustic soundscapes dynamically interact. This comprehensive document provides an exhaustive technical analysis of all five studio tabs, underlying ML algorithms, mathematical formulations, a dedicated 20-question Q&A guide for hackathon judges, and our complete 'Vibe Coding' agentic methodology.

1. Comprehensive Technical Stack Matrix

Component Layer	Technologies & Libraries	Architectural Responsibility
Frontend & UX	Streamlit 1.32+, Custom Dark Obsidian CSS, p5.js (HTML5 Canvas)	Ultra-responsive 5-tab reactive interface, dynamic flow field physics, live audio controllers.
Multimodal Vision	In-house Multimodal Vision Engine (Gemini Flash REST API)	Zero-shot visual inspection: Context extraction, mood deduction, symbolic intent & living narrative generation.
Deep Salient Matting	U2NetP, ISNet (ONNX Runtime), OpenCV GrabCut (GMMs)	Cloud-safe salient object segmentation, sub-pixel Gaussian feathering, custom bokeh & studio gradients.
Diffusion Synthesis	Prompt Engineering Engine, Pollinations Diffusion	Cinematographic prompt elevation, auto-retry cooldown handling, direct canvas interchange.
Optical Transformation	NumPy, PIL (Pillow), OpenCV (cv2.bilateralFilter)	Color temperature grading (Cold-Warm), dynamic chaos grading, 6 artistic neural presets.
Acoustics & Voice	Web Audio API (Synthesizers), Web Speech API (TTS)	BiquadFilter acoustic noise, celestial harmonic drones, multi-lingual audio narration.
Persistence & Export	SQLite3 (sudarshan_memory.db), ReportLab, Plotly	Session trajectory archival, 2D emotional timeline graph, XML-safe PDF storybook generation.

2. Deep Tab-by-Tab Architectural Breakdown

Tab 1: Vision Context & Storyteller Studio

Workflow: Image → Base64 serialization → Multimodal Vision Inspection → Structured JSON extracting Context, Mood, Conveying Message, and Living Story. Continuous emotional coordinates (Valence in [-1, 1], Arousal in [0, 1]) are calculated.

- **Live Reactive p5.js Canvas:** Driven by Valence and Arousal. Low valence & arousal triggers slow, cold indigo laminar flow (0.35x velocity). High valence & arousal triggers turbulent golden starlight particles (1.8x velocity).
- **Multilingual Lore Engine:** Generates narratives in English, Hindi, or Hinglish with browser-native Web Speech TTS narration and mood-matched ambient synth soundscapes.

Tab 2: AI Image Generator (Prompt Studio)

Workflow: Text Prompt → Neural Prompt Elevation → Diffusion Engine → Watermark-Free Direct Canvas Integration.

- **In-House Prompt Elevation:** Short queries are expanded into 40-word cinematographic descriptions specifying lighting, volumetric rays, 8k texture, and camera focal length.
- **Auto-Retry Cooldown Handler:** Clean URL routing eliminates HTTP 402 paid flags. Built-in backoff parses retry cooldowns automatically, ensuring zero-error synthesis.

Tab 3: Prompt-Guided Image Enhancer

Workflow: Operates directly on the *exact uploaded image* without hallucinating a completely new subject. Applies spatial pixel transformations: RGB channel gains (warming reds/yellows), unsharp masking, and contrast stretching.

Tab 4: Sudarshan Studio Cutout (Deep Salient Matting)

Workflow: Image → Memory Guard Downsampling (≤ 1024px) → Neural Matting → Sub-Pixel Alpha Feathering → Backdrop Compositing.

- **Multi-Engine Selection:** Fast AI (U2NetP, 4.7MB lightweight, cloud-safe), Studio AI (ISNet general use), and Classical GrabCut (OpenCV GMM segmentation).
- **Sub-Pixel Edge Feathering:** Gaussian anti-aliasing on the alpha channel eliminates jagged pixel borders without costly Laplacian solvers.

Tab 5: Mood & Style Transformer

Workflow: Optical sliders (Temperature: Cold ↔ Warm, Dynamic: Soft ↔ Chaotic) + 6 Artistic Presets (Cinematic Teal & Orange, Film Noir, Golden Hour, Cyberpunk Neon, Impressionist Oil Painting, and Fine Pencil Sketch).

3. Core Machine Learning & Computer Vision Foundations

A. Russell's Circumplex Model of Affect (Valence & Arousal)

Emotion is mapped not to discrete labels, but to a continuous 2D coordinate space: **Valence** ([-1, 1]: displeasure to pleasure) and **Arousal** ([0, 1]: calm to excitement). Joy = (+0.8, 0.7), Sadness = (-0.8, 0.2), Anger = (-0.7, 0.9), Wonder = (+0.7, 0.8). This continuous mapping directly drives physical variables: particle velocity $v = v0 \times (1 + \text{arousal})$, color hue $\theta = f(\text{valence})$, and audio oscillator frequencies.

B. Deep Salient Object Detection (U2-Net & IS-Net)

Unlike semantic segmentation, Salient Object Detection (SOD) identifies the dominant subject. U2Net employs ReSidual U-blocks (RSU) that allow nested multi-scale feature extraction at every stage without reducing spatial feature map resolution. U2NetP is a pruned 4.7MB architecture with 1.1M parameters, enabling real-time 40ms CPU inference.

C. OpenCV GrabCut Segmentation Algorithm

GrabCut formulates foreground extraction as energy minimization over a graph cut. Color distributions are modeled via two 5-component Gaussian Mixture Models (GMMs). A min-cut/max-flow algorithm finds the optimal boundary where color contrast is highest.

D. Edge-Preserving Bilateral Filtering (Oil Painting Preset)

Standard Gaussian blur blurs edges. Bilateral filtering replaces each pixel's intensity with a weighted average of nearby pixels, where weights depend on both spatial distance and photometric intensity difference. This flattens surface textures while maintaining sharp silhouettes.

4. The 'Vibe Coding' Architecture & Methodology

Definition: Vibe Coding (Andrej Karpathy, 2025) is an advanced software engineering paradigm where the human developer acts as the Systems Architect and Creative Director, orchestrating an autonomous AI coding agent (Antigravity) through a continuous **Architecture** ↔ **Tool Execution** ↔ **Runtime Feedback Loop**. Rather than copy-pasting code, the developer and agent collaboratively scaffold, diagnose, profile, and deploy complex production systems at 10x speed.

Four Real Production Bottlenecks Solved via Vibe Coding:

- **1 GB Container Memory (OOM) Optimization:** Diagnosed process crash from terminal logs after ISNet model loading; replaced memory-heavy Laplacian solvers with pruned U2NetP (4.7MB) and Gaussian alpha anti-aliasing (<35MB RAM).
- **Eliminating Pip C-Compilation Backtracking:** Traced a 15-minute container freeze on 'Building wheel for pandas' to an unneeded scikit-learn constraint; purged the library and unpinned NumPy, cutting build time from 15 minutes to 12 seconds.
- **Solving 45s Cooldowns & HTTP 402 Errors:** Isolated paid query flags (?nologo=true) on public diffusion clusters; implemented exponential backoff regex retry parsing.
- **Streamlit Startup Order & XML Hardening:** Hoisted st.set_page_config to line 20 before custom component imports; escaped XML entities (&, <) in ReportLab to prevent parser crashes.

Winning Response for Judges ('How Did You Vibe Code This?'):

"We built Sudarshan Studio through an active agentic vibe-coding loop. I directed the system architecture, mathematical emotion mapping (Russell's Circumplex Model), and user experience, while our AI pair-programming agent wrote modular components, compiled code, monitored runtime terminal stdout/stderr, diagnosed container memory limits, and pushed commits directly to GitHub for continuous deployment. It represents true collaborative human-AI software engineering."

5. Top 20 Judge & Viva Questions with High-Scoring Answers

Q1: What is the core innovation of Sudarshan Studio compared to standard AI wrappers?

Answer: Most AI tools generate isolated static images from prompts. In Sudarshan Studio, the *system's behavior is the message*. Emotion coordinates (Valence and Arousal) actively modulate the entire visual pipeline—particle physics speed, color temperature, sound harmonics, camera framing, and narrative branching all react in real-time to mirror human sentiment.

Q2: How does the system extract emotion from uploaded images?

Answer: We utilize a Multimodal Vision-Language architecture. The image is serialized to Base64 and processed through vision cross-attention layers. We enforce a strict JSON schema prompt returning four structured fields: Context (physical setting), Mood (emotional tone), Conveying Message (underlying subtext), and continuous Valence (-1 to +1) / Arousal (0 to 1) coordinates.

Q3: Explain the difference between discrete emotion labels and Russell's Circumplex Model.

Answer: Discrete labels (e.g. 'happy', 'sad') are binary and rigid. Russell's Circumplex Model represents affective state as continuous coordinates in a 2D Euclidean space (Valence on X, Arousal on Y). This enables mathematical continuity: as valence transitions from -0.8 to +0.8, particle velocities, audio frequencies, and color gradients interpolate smoothly rather than jumping abruptly.

Q4: How does your background removal pipeline work without crashing on cloud RAM limits?

Answer: Free cloud containers (like Streamlit Community Cloud) have a strict 1 GB RAM limit. We implemented a 3-layer architecture: (1) Image downscaling to 1024px before neural matting; (2) Defaulting to U2NetP (a 4.7MB pruned network using ~35MB RAM); (3) Disabling PyMatting's memory-heavy sparse Laplacian solver in favor of fast Gaussian alpha anti-aliasing; (4) Automatic fallback to OpenCV GrabCut.

Q5: How does U2Net achieve high-quality boundary segmentation with low parameters?

Answer: Traditional networks use pre-trained backbones (like ResNet) which lose high-resolution details during repeated pooling. U2Net introduces ReSidual U-blocks (RSU) which are small U-nets nested inside each stage of a larger U-structure. This allows multi-scale feature extraction directly at the native resolution without downsampling.

Q6: What algorithm does GrabCut use, and when does it fail?

Answer: GrabCut uses Gaussian Mixture Models (5 components each for foreground and background) and solves a min-cut/max-flow graph cut problem. It excels on high-contrast objects with clear boundaries, but can struggle with thin hair, semi-transparent glass, or when the subject has identical color distributions to the background.

Q7: How does your Prompt Image Enhancer modify an image without generating a new hallucinated subject?

Answer: Rather than running full latent diffusion that regenerates the image from Gaussian noise, the Prompt Image Enhancer performs targeted optical and matrix transformations on the existing pixel tensor. It adjusts RGB gain curves, saturation matrices, and contrast curves based on prompt intent (e.g. warming reds and yellows for 'golden hour').

Q8: How does your in-house Neural Prompt Elevation work?

Answer: User prompts are often brief (e.g. 'sunset castle'). Our cinematographic elevation engine prompts the LLM with a specialized system instruction specifying camera lens (e.g. 85mm f/1.4), volumetric god rays, subsurface scattering, color grading palette, and 8k octane render keywords, boosting the visual quality of the resulting diffusion output.

Q9: How do you handle third-party API rate limits and cooldowns?

Answer: We stripped all paid URL query parameters that trigger HTTP 402 rejections. When the public cluster enters its 45-second cooldown, our generator parses the cooldown time, waits via exponential backoff, and retries automatically. If an outage occurs, it gracefully renders an emotion-matched procedural art canvas.

Q10: What is the role of Web Audio API in expressing emotion?

Answer: Sound reinforces the emotional state. We use Web Audio API oscillators and BiquadFilters to synthesize procedural soundscapes in real-time. For melancholy, we generate low-pass filtered pink noise simulating raindrops with sub-bass drones. For joy, we synthesize radiant major harmonies (E4/B4 chords).

Q11: How do you guarantee character and visual consistency across story chapters?

Answer: In story.py, character attributes (e.g. 'Babu the Clockwork Sparrow with burnished brass wings') and setting descriptors are locked in st.session_state. Every successive narrative and image prompt injects these locked tokens, ensuring the hero maintains consistent visual identity across scenes.

Q12: How does the p5.js canvas communicate with Streamlit?

Answer: We use Streamlit components (st.components.v1.html) to inject a standalone responsive p5.js script. Current Valence and Arousal coordinates are embedded dynamically into the canvas initialization script, configuring particle count (120), velocity multiplier, turbulence noise scale, and color hex vectors.

Q13: How does the Impressionist Oil Painting preset work under the hood?

Answer: It applies an edge-preserving OpenCV Bilateral Filter (diameter=9, sigmaColor=75, sigmaSpace=75) which smooths pixel gradients in uniform regions while keeping edges crisp. We then boost color saturation by 35% using PIL ImageEnhance.Color to mimic oil paint pigments.

Q14: How does the Fine Pencil Sketch filter work?

Answer: It converts the image to grayscale, inverts the color values, applies an 8-pixel Gaussian blur to create a negative mask, and then performs a Color Dodge blend operation: Dodge(A, B) = min(255, (A * 256) / (255 - B + 1)). This extracts fine pencil contour lines.

Q15: How is session history persisted, and how can it be replayed?

Answer: We use a lightweight SQLite database (sudarshan_memory.db) managed by storage.py. Each session saves timestamp, prompt history, valence/arousal journey points, and scene narratives. Users can query past sessions and export a publication-ready PDF storybook.

Q16: How do you prevent XML injection in ReportLab PDF generation?

Answer: ReportLab Paragraph objects parse XML-like tags (like <b> and <i>). If a user prompt contains '&' or '<', ReportLab throws an unhandled crash. We wrap all dynamic user and narrative strings in html.escape() before instantiating Paragraph objects.

Q17: Why did you choose SQLite over NoSQL or cloud databases?

Answer: SQLite is serverless, zero-configuration, embeds directly inside the project directory, and requires no external network credentials. For an interactive studio session tracker, it provides ACID-compliant storage with sub-millisecond query latency.

Q18: How is multilingual storytelling handled for Hindi and Hinglish?

Answer: The LLM prompt dynamically specifies the output dialect. For Hinglish, it uses colloquial Roman Hindi combined with conversational English. Web Speech API detects the language tag and assigns 'hi-IN' or 'en-US' voice synthesis engines accordingly.

Q19: What is the computational complexity of the background removal models?

Answer: GrabCut has an O(V × E) complexity per iteration where V is pixel count and E is graph edges. Neural SOD (U2NetP) has a deterministic feedforward complexity of O(C × H × W) requiring ~1.2 GFLOPs, executing in under 50ms on standard cloud CPUs.

Q20: What are your key future architectural improvements for scaling Sudarshan Studio?

Answer: (1) Integrating Depth-Anything-V2 for real-time 3D parallax mesh displacement; (2) Fine-tuned LoRA weights for personalized avatar style locking; (3) WebGL/Three.js shader pipelines for GPU-accelerated real-time cinematic post-processing in the browser.